# Network Utilities & Service Enumeration Lab

Practical Lab Report

| Item | Detail |
|---|---|
| Platform | Kali Linux |
| Target | <TARGET_IP> (address hidden) |
| Tools | Ping, Traceroute, Nmap, Telnet, Netcat, Curl |
| Category | Network Reconnaissance |

**Note:** IP addresses and port numbers have been replaced with placeholders (<TARGET_IP>, <HTTP_PORT>, <FTP_PORT>, <LISTENER_PORT>) so the report can be shared safely. The full list of commands is in the accompanying README.md.

## 1. Objective

This lab introduced the essential network utilities used during the reconnaissance and service enumeration phases of penetration testing. We tested connectivity, traced network paths, identified services, collected service banners, and practiced basic TCP client-server communication.

## 2. Environment Preparation

A dedicated working directory was created to keep the lab organized, with separate folders for scan results (**scans**), practical evidence (**evidence**), and documentation (**reports**). The structure was verified afterwards with `pwd` and `ls -R`.

## 3. Ping — ICMP Connectivity Testing

Ping uses ICMP Echo Requests to check whether the target responds. Five packets were sent to <TARGET_IP>. We examined the main output fields: the sequence number (*icmp_seq*), the Time To Live (*ttl*), the round-trip *time*, and the percentage of *packet loss*.

> **Key takeaway:** a failed ping does not necessarily mean the host is offline, because a firewall may block ICMP traffic.

## 4. Traceroute — Network Path Discovery

Traceroute lists the hops between our machine and the target by sending packets with progressively increasing TTL values. Each hop shows its position, the responding device's address, and the response time; an asterisk means no reply arrived before the timeout.

We then ran a **TCP traceroute** against <HTTP_PORT>. Unlike the default UDP method, TCP packets are more likely to pass through firewalls that allow web traffic, which can reveal more of the path.

## 5. Nmap — Port and Service Detection

Nmap was used to check the HTTP port on the target. A service-version scan (`-sV`) was followed by a SYN scan with `-Pn` (skip host discovery) and `--reason` (explain the port state).

| Port state | Meaning |
|---|---|
| open | A service is accepting connections |
| closed | The host responds, but nothing is listening on the port |
| filtered | Nmap cannot tell whether the port is open or closed (filtering or no response) |

**Observed result:** <HTTP_PORT>/tcp was reported as **filtered** (http). The scan could not determine whether the port was open or closed. Possible causes are firewall rules, network filtering, or missing responses.

## 6. Telnet — HTTP Banner Grabbing

Telnet was used as a simple TCP client to connect to the HTTP port and type a request by hand (a `GET /` request with a Host header, finished by a blank line). From the response we would inspect

the status code, server information, Content-Type, and other headers. To leave the session: `Ctrl + ]`, then `quit`.

## 7. Netcat — HTTP Banner Grabbing

Netcat achieved the same goal as Telnet with more flexibility. After connecting to <TARGET_IP>:<HTTP_PORT>, a manual HTTP request was sent.

> **Key takeaway:** establishing a TCP connection does not mean the server will answer. An appropriate application-layer request is usually required.

## 8. FTP Banner Grabbing

FTP uses a control connection (<FTP_PORT>) and normally sends a welcome banner as soon as a client connects. Connecting with Netcat lets us read that banner, which may reveal the FTP software, its version, and the initial service response.

## 9. Netcat Listener and Client

Two terminals on the local Kali machine were used. The first ran Netcat in listening mode on <LISTENER_PORT>; the second connected to it through the localhost address. A test message typed in one terminal appeared in the other, and the opposite direction was tested too. This demonstrated TCP listener setup, client connections, and two-way text communication.

> **Security note:** this was ordinary TCP text communication, not a reverse shell or remote command execution.

## 10. Curl — HTTP Request Testing

Curl with the `-i` option sends an HTTP request and shows the response headers with the body. Common status codes reviewed:

| Status | Meaning |
|---|---|
| 200 OK | Request succeeded |
| 301 | Permanent redirection |
| 302 | Temporary redirection |
| 403 | Access forbidden |
| 404 | Resource not found |
| 500 | Internal server error |

## 11. Evidence and Documentation

Output from the tools was saved into the prepared folders: Nmap results in **scans/** (using `-oN`), and the Ping and Traceroute output in **evidence/** (using redirection).

## 12. Skills Summary

| Tool | Practical skill |
|---|---|
| Ping | ICMP connectivity testing |
| Traceroute | Network path discovery |
| TCP Traceroute | TCP-based path tracing |
| Nmap | Port scanning and service detection |
| Telnet | Manual TCP connections and HTTP requests |
| Netcat | TCP connections and banner grabbing |
| FTP banner | Reading FTP service banners |
| Netcat listener | TCP listener / client communication |
| Curl | HTTP request and response inspection |

## 13. Conclusion

This lab gave hands-on experience with the fundamental utilities used in reconnaissance and enumeration: ICMP testing, TTL and path discovery, TCP connections, port states, service identification, banner grabbing, and basic client-server communication. These skills are a foundation for junior penetration testers, helping them find accessible services and understand how applications respond before deeper security assessment.